

```

import { useState } from 'react';
import { motion, AnimatePresence } from 'framer-motion';
import { Sidebar } from './components/Sidebar';
import { NowPlaying } from './components/NowPlaying';
import { Library } from './components/Library';
import { PlayerBar } from './components/PlayerBar';
import { MobileTabs } from './components/MobileTabs';
import { useProgress } from './hooks/useProgress';

type View = 'home' | 'library';

function App() {
  const [currentView, setCurrentView] = useState<View>('home');
  useProgress();

  return (
    <div className="flex flex-col h-screen w-full bg-[var(--bg-main)] text-[var(--text-main)]
overflow-hidden font-sans transition-colors duration-500 relative">
      {/* 1. CUERPO SUPERIOR (Sidebar + Contenido) */}
      <div className="flex flex-1 overflow-hidden relative">
        {/* SIDEBAR: Solo visible en escritorio */}
        <div className="hidden md:flex h-full shrink-0 border-r border-[var(--glass-border)]">
          <Sidebar setView={setCurrentView} currentView={currentView} />
        </div>

        <main className="flex-1 relative overflow-hidden bg-[var(--bg-main)]">
          <AnimatePresence mode="sync">
            {currentView === 'home' ? (
              <motion.div
                key="now-playing"
                initial={{ y: '100%' }}
                animate={{ y: 0 }}
                exit={{ y: '100%' }}
                transition={{ type: 'spring', damping: 25, stiffness: 200 }}
                className="absolute inset-0 z-50 bg-[var(--bg-main)]"
              >
                <NowPlaying setView={setCurrentView} />
              </motion.div>
            ) : (
              <motion.div

```

```

    key="library"
    initial={{ opacity: 0 }}
    animate={{ opacity: 1 }}
    exit={{ opacity: 0 }}
    /* pb-60 asegura que el scroll no quede tapado por los controles en móvil */
    className="h-full overflow-y-auto custom-scrollbar pb-60 md:pb-0"
  >
    <Library />
  </motion.div>
)}
</AnimatePresence>
</main>
</div>

```

```

{ /* 2. ZONA DE CONTROLES INFERIOR */ }
<div className="relative shrink-0 w-full">
  { /* PLAYERBAR: Flotante sobre tabs en móvil, fija al fondo en desktop */ }
  <AnimatePresence>
    {currentView !== 'home' && (
      <motion.div
        initial={{ y: 100, opacity: 0 }}
        animate={{ y: 0, opacity: 1 }}
        exit={{ y: 100, opacity: 0 }}
        className={`
          fixed left-0 right-0 px-3 z-40 pointer-events-auto
          /* Móvil: Flota arriba de las tabs (h-16 = 4rem) + safe area + margen */
          bottom-[calc(4rem+env(safe-area-inset-bottom)+12px)]
          /* Desktop: Se integra al flujo normal al final de la pantalla */
          md:relative md:bottom-0 md:px-0 md:bg-black md:border-t md:border-[var(--glass-
border)]
        `}
      >
        <div className="max-w-7xl mx-auto shadow-[0_-10px_40px_rgba(0,0,0,0.5)]
          md:shadow-none rounded-2xl md:rounded-none overflow-hidden border border-white/5
          md:border-none">
          <PlayerBar />
        </div>
      </motion.div>
    )}
  </AnimatePresence>

```

```

    { /* MOBILE TABS: Siempre anclada al fondo en móvil */ }
    <div className="md:hidden fixed bottom-0 left-0 right-0 h-16 z-50 bg-black/90
backdrop-blur-2xl border-t border-white/10 pb-[env(safe-area-inset-bottom)]">
      <MobileTabs currentView={currentView} setView={setCurrentView} />
    </div>
  </div>
</div>
);
}

```

export default App;

index.css:

```
@import "tailwindcss";
```

```
/* 1. CONFIGURACIÓN DE VARIABLES (Temas dinámicos) */
```

```

:root {
  --bg-main: #080808;
  --text-main: #ffffff;
  --text-muted: #6b7280;
  --glass-bg: rgba(255, 255, 255, 0.03);
  --glass-border: rgba(255, 255, 255, 0.08);
  --accent-primary: #3b82f6;
  --accent-secondary: #a855f7;

```

```
/* Evita el resaltado azul al tocar en móviles */
```

```

-webkit-tap-highlight-color: transparent;
}

```

```

:root.light {
  --bg-main: #f8fafc;
  --text-main: #0f172a;
  --text-muted: #64748b;
  --glass-bg: rgba(15, 23, 42, 0.04);
  --glass-border: rgba(15, 23, 42, 0.08);
}

```

```
/* 2. ESTILOS BASE */
```

```
body {  
margin: 0;  
background-color: var(--bg-main);  
color: var(--text-main);  
font-family: ui-sans-serif, system-ui, -apple-system, BlinkMacSystemFont, "Segoe UI",  
Roboto, sans-serif;  
-webkit-font-smoothing: antialiased;  
-moz-osx-font-smoothing: grayscale;  
transition: background-color 0.5s ease, color 0.5s ease;  
/* Importante para PWA: evita rebotes elásticos molestos en el body */  
overflow: hidden;  
height: 100vh;  
width: 100vw;  
position: fixed;  
}
```

```
/* 3. OPTIMIZACIONES DE UI */
```

```
/* Scrollbar Minimalista */
```

```
.custom-scrollbar::-webkit-scrollbar {  
width: 5px;  
}
```

```
.custom-scrollbar::-webkit-scrollbar-track {  
background: transparent;  
}
```

```
.custom-scrollbar::-webkit-scrollbar-thumb {  
background: var(--glass-border);  
border-radius: 10px;  
}
```

```
/* Ocultar scrollbar */
```

```
.no-scrollbar::-webkit-scrollbar {  
display: none;  
}
```

```
.no-scrollbar {  
-ms-overflow-style: none;  
scrollbar-width: none;
```

```
}
```

```
/* 4. UTILIDADES DE ÁREA SEGURA (Critical para móviles) */
```

```
/* Padding bottom que suma la altura de MobileTabs (64px) + el Notch + espacio extra */
```

```
.pb-mobile-controls {  
padding-bottom: calc(4rem + env(safe-area-inset-bottom) + 80px);  
}
```

```
/* Asegura que el contenido se vea bajo el notch arriba si es necesario */
```

```
.pt-safe {  
padding-top: env(safe-area-inset-top);  
}
```

```
.pb-safe {  
padding-bottom: env(safe-area-inset-bottom);  
}
```

```
/* 5. ANIMACIONES */
```

```
@keyframes fadeIn {  
from {  
    opacity: 0;  
    transform: translateY(10px);  
}
```

```
to {  
    opacity: 1;  
    transform: translateY(0);  
}  
}
```

```
.animate-view-entry {  
animation: fadeIn 0.6s cubic-bezier(0.16, 1, 0.3, 1) forwards;  
}
```

```
/* Efecto de cristal para el PlayerBar y Tabs */
```

```
.glass-effect {  
background: var(--glass-bg);  
backdrop-filter: blur(20px) saturate(180%);  
-webkit-backdrop-filter: blur(20px) saturate(180%);  
}
```

```
border: 1px solid var(--glass-border);
}
```

navigation.ts:

```
import type { LucideIcon } from 'lucide-react';
```

```
export interface NavItem {
  label: string;
  icon: LucideIcon;
  href: string;
  category?: 'main' | 'library' | 'settings';
}
```

track.ts:

```
export interface Track {
  id: string;
  title: string;
  artist: string;
  coverUrl: string;
  audioUrl: string; // Se usará para música de internet
  fileData?: File | Blob; // Se usará para música local (IndexedDB)
}
```

```
import { create } from 'zustand';
import { persist, type PersistStorage, type StorageValue } from 'zustand/middleware';
import { get, set, del } from 'idb-keyval';
import { Howl, Howler } from 'howler';
import type { Track } from '../types/track';
```

```
interface PersistedPlayerState {
  songs: Track[];
  volume: number;
}
```

```
interface HowlInternal extends Howl {
  _sounds: { _node: HTMLAudioElement }[];
}
```

```
const customBinaryStorage: PersistStorage<PersistedPlayerState> = {
```

```
getItem: async (name) => (await get<StorageValue<PersistedPlayerState>>(name)) || null,  
setItem: async (name, value) => await set(name, value),  
removeItem: async (name) => await del(name),  
};
```

```
interface PlayerState {  
  songs: Track[];  
  currentTrack: Track | null;  
  isPlaying: boolean;  
  volume: number;  
  seek: number;  
  duration: number;  
  howl: Howl | null;  
  analyzer: AnalyserNode | null;  
  addSongs: (files: File[]) => void;  
  clearSongs: () => void;  
  setTrack: (track: Track) => void;  
  togglePlay: () => void;  
  setVolume: (val: number) => void;  
  setSeek: (val: number) => void;  
  updateProgress: () => void;  
}
```

```
export const usePlayerStore = create<PlayerState>()  
persist(  
  (set, get) => ({  
    songs: [],  
    currentTrack: null,  
    isPlaying: false,  
    volume: 0.5,  
    seek: 0,  
    duration: 0,  
    howl: null,  
    analyzer: null,  
  
    addSongs: (files) => {  
      const newTracks = files.map((file) => ({  
        id: crypto.randomUUID(),  
        title: file.name.replace(/\.([^/\.]+$/), "  
        artist: 'Archivo Local',
```

```

    coverUrl: 'https://unsplash.com',
    audioUrl: "",
    fileData: file,
  }));
  set((state) => ({ songs: [...state.songs, ...newTracks] }));
},

clearSongs: () => {
  const { howl } = get();
  if (howl) howl.unload();
  set({ songs: [], currentTrack: null, isPlaying: false, analyzer: null, howl: null });
},

setTrack: (track) => {
  const { howl: oldHowl } = get();
  if (oldHowl) oldHowl.unload();
  if (!track.fileData) return;

  // Solución al error de TS: Forzamos a File para acceder a .name
  const file = track.fileData as File;
  const extension = file.name?.split('.').pop()?.toLowerCase() || 'mp3';
  const activeUrl = URL.createObjectURL(file);

  const newHowl = new Howl({
    src: [activeUrl],
    format: [extension],
    html5: true,
    volume: get().volume,
    onplay: () => {
      set({ isPlaying: true, duration: newHowl.duration() });

      // Conexión del Analizador
      const node = (newHowl as unknown as HowlInternal)._sounds?.[0]?._node;
      if (node) {
        try {
          let analyzer = get().analyzer;
          if (!analyzer) {
            const source = Howler.ctx.createMediaElementSource(node);
            analyzer = Howler.ctx.createAnalyser();
            analyzer.fftSize = 256;

```



```

        source.connect(analyzer);
        analyzer.connect(Howler.ctx.destination);
        set({ analyzer });
    }
} catch {
    console.debug('Audio node already connected');
}
},
onpause: () => set({ isPlaying: false }),
onend: () => set({ isPlaying: false, seek: 0 }),
onload: () => set({ duration: newHowl.duration() }),
});

newHowl.play();
set({ currentTrack: track, howl: newHowl, isPlaying: true });
},

togglePlay: () => {
    const { howl, isPlaying } = get();
    if (!howl) return;

    if (isPlaying) {
        howl.pause();
    } else {
        if (Howler.ctx.state === 'suspended') Howler.ctx.resume();
        howl.play();
    }
},

setVolume: (val) => {
    const { howl } = get();
    if (howl) howl.volume(val);
    set({ volume: val });
},

setSeek: (val) => {
    const { howl } = get();
    if (howl) {
        howl.seek(val);
    }
}

```

```

    set({ seek: val });
  }
},

updateProgress: () => {
  const { howl, isPlaying } = get();
  if (howl && isPlaying) {
    const s = howl.seek();
    if (typeof s === 'number') set({ seek: s });
  }
},
}),
{
  name: 'musie-pwa-storage',
  storage: customBinaryStorage,
  partialize: (state): PersistedPlayerState => ({
    songs: state.songs,
    volume: state.volume,
  })),
}
)
);

import { useEffect } from 'react';
import { usePlayerStore } from '../store/usePlayerStore';

export const useProgress = () => {
  const { updateProgress, isPlaying } = usePlayerStore();

  useEffect(() => {
    if (!isPlaying) return;

    const interval = setInterval(() => {
      updateProgress();
    }, 1000); // Se actualiza cada segundo

    return () => clearInterval(interval);
  }, [isPlaying, updateProgress]);
};

```

```
import { useEffect, useState } from 'react';
import { usePlayerStore } from '../store/usePlayerStore';

export const useVisualizer = () => {
  // Obtenemos el analizador y el estado de reproducción del Store de Zustand
  const { analyzer, isPlaying } = usePlayerStore();

  // Guardamos los datos de las frecuencias en un array de bytes (0 a 255)
  const [audioData, setAudioData] = useState<Uint8Array>(new Uint8Array(0));

  useEffect(() => {
    // Si no hay música sonando o no hay analizador, no hacemos nada
    if (!analyzer || !isPlaying) return;

    const bufferLength = analyzer.frequencyBinCount;
    const dataArray = new Uint8Array(bufferLength);

    const update = () => {
      // 1. Obtenemos los datos actuales de la frecuencia
      analyzer.getBytesFrequencyData(dataArray);

      // 2. Actualizamos el estado con una copia de los datos para que React redibuje
      setAudioData(new Uint8Array(dataArray));

      // 3. Solicitamos el siguiente frame de animación (aprox. 60fps)
      requestAnimationFrame(update);
    };

    const animationId = requestAnimationFrame(update);

    // Limpieza al desmontar el componente o pausar
    return () => cancelAnimationFrame(animationId);
  }, [analyzer, isPlaying]);

  return audioData;
};

import { useState, useRef } from 'react';
import {
```

```
Home,  
Library,  
Disc,  
ListMusic,  
Heart,  
SlidersHorizontal,  
Activity,  
Settings,  
User,  
Pin,  
Moon,  
Sun,  
type LucideIcon,  
} from 'lucide-react';
```

```
type View = 'home' | 'library';
```

```
interface NavItem {  
  icon: LucideIcon;  
  label: string;  
}
```

```
interface SidebarSection {  
  title: string;  
  items: NavItem[];  
}
```

```
export const Sidebar = ({  
  currentView,  
  setView,  
}: {  
  currentView: View;  
  setView: (v: View) => void;  
}) => {  
  const [isPinned, setIsPinned] = useState(false);  
  const [isHovered, setIsHovered] = useState(false);  
  const [isDarkMode, setIsDarkMode] = useState(true);  
  const timeoutRef = useRef<ReturnType<typeof setTimeout> | null>(null);  
  
  const handleMouseEnter = () => {
```

```
    if (timeoutRef.current) clearTimeout(timeoutRef.current);
    setIsHovered(true);
  };
```

```
const handleMouseLeave = () => {
  timeoutRef.current = setTimeout(() => setIsHovered(false), 700);
};
```

```
const toggleTheme = () => {
  setIsDarkMode(!isDarkMode);
  document.documentElement.classList.toggle('light');
};
```

```
const isExpanded = isPinned || isHovered;
```

```
const sections: SidebarSection[] = [
  {
    title: 'Space',
    items: [
      { icon: Disc, label: 'Albums' },
      { icon: ListMusic, label: 'Playlists' },
      { icon: Heart, label: 'Favorites' },
    ],
  },
  {
    title: 'Settings',
    items: [
      { icon: SlidersHorizontal, label: 'Equalizer' },
      { icon: Activity, label: 'Visualizer' },
      { icon: Settings, label: 'Preferences' },
    ],
  },
];
```

```
return (
  <aside
    onMouseEnter={handleMouseEnter}
    onMouseLeave={handleMouseLeave}
    className={`h-full flex flex-col z-40 transition-[width,background-color] duration-700
ease-in-out shrink-0 border-r
```

```

    ${isExpanded ? 'w-56' : 'w-20'}
    ${
      isDarkMode
        ? 'bg-[#050505] border-white/5 shadow-[20px_0_50px_rgba(0,0,0,0.5)]'
        : 'bg-white border-black/5 shadow-[20px_0_50px_rgba(0,0,0,0.05)]'
    }
  }
}
>
{ /* 1. LOGO Y PIN */}
<div className="h-20 flex items-center px-[26px] shrink-0 overflow-hidden">
  <div className="flex items-center justify-between min-w-[172px] w-full">
    <div className="flex items-center gap-3">
      <div
        className={`w-7 h-7 rounded-lg shrink-0 transition-all duration-500 shadow-lg
        ${
          isDarkMode
            ? 'bg-gradient-to-br from-blue-600 to-purple-600 shadow-blue-500/20'
            : 'bg-gradient-to-br from-blue-500 to-cyan-400 shadow-blue-500/40'
          }}
      />
      <h1
        className={`text-lg font-black tracking-tighter transition-all duration-500
        ${isDarkMode ? 'text-white' : 'text-black'}
        ${isExpanded ? 'opacity-100' : 'opacity-0'}}
      >
        MUSIE
      </h1>
    </div>
    <button
      onClick={() => setIsPinned(!isPinned)}
      className={`transition-all duration-500 hover:scale-110
      ${isPinned ? 'text-blue-500 rotate-45' : 'text-gray-500'}
      ${isExpanded ? 'opacity-100' : 'opacity-0 pointer-events-none'}}
    >
      <Pin size={14} fill={isPinned ? 'currentColor' : 'none'} />
    </button>
  </div>
</div>

{ /* 2. NAVEGACIÓN PRINCIPAL */}

```

```

<nav className="flex-1 space-y-5 overflow-y-auto no-scrollbar overflow-x-hidden pt-2
px-4">
  <ul className="space-y-1">
    {[('home', 'library') as View[]].map((id) => {
      const Icon = id === 'home' ? Home : Library;
      const isActive = currentView === id;
      return (
        <li
          key={id}
          onClick={() => setView(id)}
          className={`group/nav flex items-center h-11 rounded-xl cursor-pointer transition-
all duration-300 overflow-hidden
            ${
              isActive
                ? isDarkMode
                  ? 'bg-blue-600/15 text-blue-400 border border-blue-500/20'
                  : 'bg-blue-50 text-blue-600 border border-blue-200'
                : isDarkMode
                  ? 'text-gray-500 hover:bg-white/5 hover:text-white'
                  : 'text-gray-500 hover:bg-black/5 hover:text-black'
            }
          `}
        >
          <div className="flex items-center gap-4 px-3.5 min-w-[200px]">
            <Icon
              size={22}
              className={`shrink-0 transition-colors ${isActive ? 'text-blue-500' : 'group-hover/
nav:text-blue-400'}}
            />
            <span
              className={`font-black text-base tracking-tight whitespace-nowrap transition-all
duration-700
              ${isExpanded ? 'opacity-100 translate-x-0' : 'opacity-0 -translate-x-8'}}
            >
              {id === 'home' ? 'Home' : 'Library'}
            </span>
          </div>
        </li>
      );
    })}
  </ul>

```

```

{ /* SECCIONES DINÁMICAS */
{sections.map((section) => (
  <section key={section.title} className="space-y-2">
    <p
      className={`px-3.5 text-[8px] uppercase tracking-[0.3em] font-black transition-
opacity duration-500
      ${isDarkMode ? 'text-gray-600' : 'text-gray-400'}
      ${isExpanded ? 'opacity-100' : 'opacity-0'}}`
    >
      {section.title}
    </p>
    <ul className="space-y-0.5">
      {section.items.map((item) => (
        <li
          key={item.label}
          className={`flex items-center h-9 rounded-lg cursor-pointer group/item
transition-all overflow-hidden
          ${isDarkMode ? 'text-gray-500 hover:bg-white/5 hover:text-white' : 'text-gray-500
hover:bg-black/5 hover:text-black'}}`
        >
          <div className="flex items-center gap-4 px-3.5 min-w-[200px]">
            <item.icon
              size={18}
              className="shrink-0 group-hover/item:text-blue-400 transition-colors"
            />
            <span
              className={`text-[13px] font-bold whitespace-nowrap transition-all
duration-700
              ${isExpanded ? 'opacity-100 translate-x-0' : 'opacity-0 -translate-x-8'}}`
            >
              {item.label}
            </span>
          </div>
        </li>
      )})
    </ul>
  </section>
)}}
</nav>

```



```

{ /* 3. TEMA Y USUARIO */ }
<div
  className={ `shrink-0 pt-2 border-t transition-colors ${isDarkMode ? 'border-white/5' :
'border-black/5'} }` }
>
  <div className="px-4 mb-1">
    <button
      onClick={toggleTheme}
      className={ `flex items-center w-full h-10 rounded-xl transition-all overflow-hidden
group/theme
      ${isDarkMode ? 'hover:bg-white/5' : 'hover:bg-black/5'} ` }
    >
      <div className="flex items-center gap-4 px-3.5 min-w-[200px]">
        <div
          className={ `w-7 h-7 rounded-lg flex items-center justify-center shrink-0 transition-
colors
          ${isDarkMode ? 'bg-white/5' : 'bg-black/5'} ` }
        >
          {isDarkMode ? (
            <Moon size={16} className="text-blue-400" />
          ) : (
            <Sun size={16} className="text-yellow-500" />
          ) }
        </div>
        <span
          className={ `text-[13px] font-bold transition-all duration-700
          ${isDarkMode ? 'text-gray-500 group-hover/theme:text-white' : 'text-gray-500
group-hover/theme:text-black'}
          ${isExpanded ? 'opacity-100' : 'opacity-0'} ` }
        >
          {isDarkMode ? 'Dark' : 'Light'}
        </span>
      </div>
    </button>
  </div>

  <div className="p-3">
    <div
      className={ `flex items-center h-12 rounded-xl transition-all duration-700 overflow-

```

hidden cursor-pointer border

```
    ${isDarkMode ? 'bg-white/5 border-white/5 hover:bg-white/10' : 'bg-black/5 border-black/5 hover:bg-black/10'}}
  >
```

```
    <div className="flex items-center gap-4 px-[11px] min-w-[200px]">
      <div className="w-8 h-8 rounded-lg bg-gradient-to-tr from-blue-600 to-purple-600
flex items-center justify-center shrink-0 shadow-md">
```

```
        <User size={16} className="text-white" />
```

```
      </div>
```

```
      <div
```

```
        className={`flex flex-col transition-all duration-700 ${isExpanded ? 'opacity-100
translate-x-0' : 'opacity-0 -translate-x-8'}`}>
```

```
        >
```

```
          <p className={`text-xs font-black ${isDarkMode ? 'text-white' : 'text-black'}`}>
```

```
            Usuario
```

```
          </p>
```

```
          <p className="text-[8px] text-blue-400 font-black uppercase tracking-tighter">
```

```
            Pro Plan
```

```
          </p>
```

```
        </div>
```

```
      </div>
```

```
    </div>
```

```
  </div>
```

```
</div>
```

```
</aside>
```

```
);
```

```
};
```

```
import {
```

```
  Play,
```

```
  Pause,
```

```
  SkipBack,
```

```
  SkipForward,
```

```
  Repeat,
```

```
  Shuffle,
```

```
  Volume2,
```

```
  VolumeX,
```

```
  Heart,
```

```
} from 'lucide-react';
```

```

import { usePlayerStore } from '../store/usePlayerStore';

export const PlayerBar = () => {
  const { currentTrack, isPlaying, togglePlay, seek, duration, setSeek, volume, setVolume } =
    usePlayerStore();

  if (!currentTrack) return null;

  const formatTime = (s: number) => {
    const mins = Math.floor(s / 60) || 0;
    const secs = Math.floor(s % 60) || 0;
    return `${mins}:${secs.toString().padStart(2, '0')}`;
  };

  const progressPercent = (seek / duration) * 100 || 0;

  return (
    /*
      Contenedor principal:
      - En móvil: bg-black/80 con blur para que destaque sobre el contenido.
      - En desktop: transparente porque App.tsx ya le da el fondo sólido.
    */
    <div className="w-full h-16 md:h-24 px-4 md:px-6 flex items-center justify-between
      transition-all duration-500 bg-black/80 md:bg-transparent backdrop-blur-xl md:backdrop-
      blur-0">
      {/* 1. INFO CANCIÓN */}
      <div className="flex items-center gap-3 md:gap-4 w-[60%] md:w-[30%] min-w-0">
        <div className="relative shrink-0 group">
          <img
            src={currentTrack.coverUrl}
            alt={currentTrack.title}
            className="w-10 h-10 md:w-14 md:h-14 rounded-lg object-cover shadow-lg border
            border-white/10"
            />
        </div>
        <div className="flex flex-col truncate">
          <h4 className="text-[var(--text-main)] text-sm md:text-base font-bold truncate">
            {currentTrack.title}
          </h4>
          <p className="text-[var(--text-muted)] text-[10px] md:text-xs font-medium uppercase

```

```

tracking-wider truncate">
    {currentTrack.artist}
  </p>
</div>
<button className="text-[var(--text-muted)] hover:text-red-500 transition-colors ml-2
hidden lg:block">
  <Heart size={18} />
</button>
</div>

```

```

{ /* 2. CONTROLES (Móvil: Solo Play/Pause | Desktop: Full) */ }
<div className="flex flex-col items-center flex-1 max-w-[45%] gap-2">
  <div className="flex items-center gap-4 md:gap-8">
    <button className="text-[var(--text-muted)] hover:text-blue-500 transition-colors
hidden md:block">
      <Shuffle size={16} />
    </button>

    <button className="text-[var(--text-main)] opacity-70 hover:opacity-100 transition-all
active:scale-90 hidden md:block">
      <SkipBack size={24} fill="currentColor" />
    </button>

    <button
      onClick={togglePlay}
      className="w-10 h-10 md:w-12 md:h-12 flex items-center justify-center bg-white
text-black rounded-full hover:scale-105 transition-all shadow-xl active:scale-95"
    >
      {isPlaying ? (
        <Pause size={20} fill="currentColor" />
      ) : (
        <Play size={20} fill="currentColor" className="ml-0.5" />
      )}
    </button>

    <button className="text-[var(--text-main)] opacity-70 hover:opacity-100 transition-all
active:scale-90 hidden md:block">
      <SkipForward size={24} fill="currentColor" />
    </button>

```

```

        <button className="text-[var(--text-muted)] hover:text-blue-500 transition-colors
hidden md:block">
            <Repeat size={16} />
        </button>
    </div>

    {/* BARRA DE PROGRESO DESKTOP */}
    <div className="w-full hidden md:flex items-center gap-4 group/progress">
        <span className="text-[10px] font-mono text-[var(--text-muted)] w-10 text-right
tabular-nums">
            {formatTime(seek)}
        </span>
        <div className="flex-1 h-1.5 bg-white/10 rounded-full relative overflow-hidden group-
hover/progress:h-2 transition-all">
            <input
                type="range"
                min="0"
                max={duration || 0}
                step="0.1"
                value={seek}
                onChange={(e) => setSeek(parseFloat(e.target.value))}
                className="absolute inset-0 w-full h-full opacity-0 z-20 cursor-pointer"
            />
            <div
                className="absolute inset-y-0 left-0 bg-blue-500 shadow-
[0_0_10px_rgba(59,130,246,0.5)]"
                style={{ width: `${progressPercent}%` }}
            />
            </div>
            <span className="text-[10px] font-mono text-[var(--text-muted)] w-10 tabular-nums">
                {formatTime(duration)}
            </span>
        </div>
    </div>

    {/* 3. VOLUMEN (SOLO DESKTOP) */}
    <div className="hidden md:flex items-center justify-end gap-3 w-[25%] group/vol">
        <button
            onClick={() => setVolume(volume === 0 ? 0.8 : 0)}
            className="text-[var(--text-muted)] hover:text-white"

```

```

>
    {volume === 0 ? <VolumeX size={18} /> : <Volume2 size={18} />}
</button>
<div className="relative w-24 h-1 bg-white/10 rounded-full overflow-hidden">
  <input
    type="range"
    min="0"
    max="1"
    step="0.01"
    value={volume}
    onChange={(e) => setVolume(parseFloat(e.target.value))}
    className="absolute inset-0 w-full h-full opacity-0 z-20 cursor-pointer"
  />
  <div
    className="h-full bg-white group-hover/vol:bg-blue-500 transition-colors"
    style={{ width: `${volume * 100}%` }}
  />
</div>
</div>

{/* BARRA DE PROGRESO MINI (Móvil - Al borde inferior del bar) */}
<div className="md:hidden absolute bottom-0 left-0 right-0 h-[2px] bg-white/5">
  <div
    className="h-full bg-blue-500 transition-all duration-300"
    style={{ width: `${progressPercent}%` }}
  />
</div>
</div>
);
};

import {
  Heart,
  MoreHorizontal,
  Play,
  Pause,
  SkipBack,
  SkipForward,
  Repeat,
  Shuffle,

```

```

ChevronDown,
Volume1,
Volume2,
VolumeX,
} from 'lucide-react';
import { usePlayerStore } from '../store/usePlayerStore';
import { useVisualizer } from '../hooks/useVisualizer';

interface NowPlayingProps {
  setView: (view: 'home' | 'library') => void;
}

export const NowPlaying = ({ setView }: NowPlayingProps) => {
  const { currentTrack, isPlaying, togglePlay, seek, duration, setSeek, volume, setVolume } =
    usePlayerStore();

  const audioData = useVisualizer();

  if (!currentTrack) return null;

  const formatTime = (s: number) =>
    `${Math.floor(s / 60)}:${Math.floor(s % 60)
      .toString()
      .padStart(2, '0')}`;

  const progressPercent = (seek / duration) * 100 || 0;

  return (
    <div className="flex-1 h-full flex flex-col items-center justify-between p-4 md:p-10 relative
overflow-hidden transition-colors duration-500 bg-[var(--bg-main)]">
      {/* 1. FONDO DINÁMICO (Blur Profundo) */}
      <div className="absolute inset-0 -z-10 opacity-20 blur-[140px] scale-150 transition-all
duration-1000">
        <img src={currentTrack.coverUrl} alt="" className="w-full h-full object-cover" />
      </div>

      <div className="max-w-4xl w-full h-full flex flex-col justify-between items-center gap-2">
        {/* 2. CABECERA (Transparente) */}
        <div className="w-full flex justify-between items-center shrink-0 px-2 py-2">
          <button

```

```

        onClick={() => setView('library')}
        className="p-2 -ml-2 text-[var(--text-main)] opacity-40 hover:opacity-100 transition-
all active:scale-90"
    >
        <ChevronDown size={32} strokeWidth={2.5} />
    </button>
    <span className="text-[10px] uppercase tracking-[0.5em] text-[var(--text-main)]
opacity-30 font-black">
        Now Playing
    </span>
    <button className="text-[var(--text-main)] opacity-40 hover:opacity-100 transition-
colors">
        <MoreHorizontal size={24} />
    </button>
</div>

```

```

{ /* 3. PORTADA GIGANTE (Efecto Glass) */ }
<div className="relative group flex-1 flex items-center justify-center min-h-0 w-full px-4
my-2">
    <div className="absolute inset-0 bg-blue-600/5 rounded-full blur-[100px]" />
    <img
        src={currentTrack.coverUrl}
        alt={currentTrack.title}
        className="h-full max-h-[380px] md:max-h-[550px] aspect-square object-cover
rounded-[48px] shadow-[0_40px_80px_rgba(0,0,0,0.4)] border border-[var(--glass-border)]
transition-transform duration-700 hover:scale-[1.02]"
    />
</div>

```

```

{ /* 4. INFO PISTA CENTRADA (Tipografía Fluida) */ }
<div className="w-full flex items-center justify-center px-8 shrink-0 relative py-2">
    <div className="flex flex-col items-center text-center max-w-[80%]">
        <h2 className="text-[clamp(1.25rem,4.5vw,1.9rem)] font-black text-[var(--text-main)]
tracking-tighter leading-tight truncate w-full">
            {currentTrack.title}
        </h2>
        <p className="text-[clamp(0.75rem,1.5vw,0.9rem)] text-blue-500 font-bold tracking-
[0.25em] uppercase mt-1 opacity-80">
            {currentTrack.artist}
        </p>
    </div>

```



```

</div>
<button className="absolute right-0 text-gray-500 hover:text-red-500 active:scale-125
hidden md:block">
  <Heart size={28} className="hover:fill-red-500 transition-colors" />
</button>
</div>

{/* 5. VISUALIZER CON GRADIENTE VERTICAL */}
<div className="w-full px-8 shrink-0">
  <div className="relative h-14 md:h-18 flex items-end justify-center gap-[3px] mb-3">
    <input
      type="range"
      min="0"
      max={duration || 0}
      step="0.1"
      value={seek}
      onChange={(e) => setSeek(parseFloat(e.target.value))}
      className="absolute inset-0 w-full h-full opacity-0 z-30 cursor-pointer"
    />
    {Array.from(audioData)
      .slice(0, 50)
      .map((v, i) => {
        const isPlayed = (i / 50) * 100 < progressPercent;
        return (
          <div
            key={i}
            className={`flex-1 rounded-full transition-all duration-75 ${
              isPlayed
                ? 'bg-gradient-to-t from-blue-600 to-purple-500 shadow-[0_0_15px_rgba(37,99,235,0.2)]'
                : 'bg-gray-500/20 opacity-30'
            }`}
            style={{ height: `${Math.max(20, (v / 255) * 100)}%` }}
          />
        );
      })}
  </div>
  <div className="flex justify-between items-center px-1 font-mono text-xs md:text-sm
text-gray-500 font-bold tracking-tighter">
    <span className="w-12">{formatTime(seek)}</span>

```

```

<div className="h-[1px] flex-1 mx-6 bg-gray-500/10 relative">
  <div
    className="absolute inset-y-0 left-0 bg-blue-500/30"
    style={{ width: `${progressPercent}%` }}
  />
</div>
<span className="w-12 text-right">{formatTime(duration)}</span>
</div>
</div>

```

```

{/* 6. CONTROLES Y VOLUMEN SOBREAMPLIFICADO (Glass Style) */}
<div className="w-full max-w-sm flex flex-col gap-6 pb-6 shrink-0 mt-4">
  <div className="flex items-center justify-between px-4">
    <button className="text-gray-500 hover:text-blue-500">
      <Shuffle size={20} />
    </button>
    <div className="flex items-center gap-8">
      <button className="text-[var(--text-main)] opacity-60 hover:opacity-100
active:scale-90">
        <SkipBack size={32} fill="currentColor" />
      </button>
      <button
        onClick={togglePlay}
        className="w-16 h-16 flex items-center justify-center bg-[var(--text-main)] text-
[var(--bg-main)] rounded-full shadow-2xl hover:scale-105 active:scale-95 transition-all"
      >
        {isPlaying ? (
          <Pause size={30} fill="currentColor" />
        ) : (
          <Play size={30} fill="currentColor" className="ml-1" />
        )}
      </button>
      <button className="text-[var(--text-main)] opacity-60 hover:opacity-100
active:scale-90">
        <SkipForward size={32} fill="currentColor" />
      </button>
    </div>
    <button className="text-gray-500 hover:text-blue-500">
      <Repeat size={20} />
    </button>
  </div>

```

</div>

{/* VOLUMEN TÁCTIL (0-200%) CON BOTÓN DE RESET 100% */}

<div className="flex items-center gap-4 px-4 group/vol">

<button

onClick={() => setVolume(volume === 0 ? 0.5 : 0)}

className="text-gray-500 hover:text-blue-500 shrink-0"

>

{volume === 0 ? <VolumeX size={22} /> : <Volume1 size={22} />}

</button>

<div className="flex-1 relative h-10 flex items-center">

<input

type="range"

min="0"

max="2"

step="0.01"

value={volume}

onChange={(e) => setVolume(parseFloat(e.target.value))}

className="absolute inset-0 w-full h-full opacity-0 z-20 cursor-pointer"

/>

<div className="flex-1 h-2 bg-gray-500/10 rounded-full overflow-hidden relative border border-[var(--glass-border)]">

<div className="absolute left-1/2 top-0 bottom-0 w-0.5 bg-white/20 z-10" />

<div

className={`h-full transition-all \${

volume > 1

? 'bg-gradient-to-r from-blue-600 via-purple-600 to-red-500'

: 'bg-gradient-to-r from-blue-700 to-blue-400'

}`}

style={{ width: `\${(volume / 2) \* 100}%` }}

/>

</div>

<div

className={`absolute w-4 h-4 rounded-full shadow-xl pointer-events-none transition-transform \${volume > 1 ? 'bg-red-500' : 'bg-[var(--text-main)]'}`}

style={{ left: `calc(\${(volume / 2) \* 100}% - 8px)` }}

/>

</div>

```

    <button
      onClick={() => setVolume(1.0)}
      className="text-gray-500 hover:text-blue-500 shrink-0 transition-all active:scale-90"
    >
      <Volume2 size={22} />
    </button>

    <span
      className={`text-xs font-mono w-12 text-right font-black transition-colors ${volume
> 1 ? 'text-red-500' : 'text-gray-500'}`}
    >
      {Math.round(volume * 100)}%
    </span>
  </div>
</div>
</div>
</div>
);
};

```

```
import { Home, Library, SlidersHorizontal, Settings } from 'lucide-react';
```

```

interface MobileTabsProps {
  currentView: 'home' | 'library';
  setView: (view: 'home' | 'library') => void;
}

```

```

export const MobileTabs = ({ currentView, setView }: MobileTabsProps) => {
  const tabs = [
    { id: 'home', icon: Home, label: 'Home' },
    { id: 'library', icon: Library, label: 'Library' },
    { id: 'equalizer', icon: SlidersHorizontal, label: 'EQ' }, // <--- Nuevo Tab
    { id: 'settings', icon: Settings, label: 'Settings' },
  ] as const;

```

```

  return (
    <nav className="md:hidden fixed bottom-0 left-0 right-0 h-14 bg-black/80 backdrop-
blur-2xl border-t border-white/5 px-6 flex items-center justify-between z-50 pb-safe">
      {tabs.map((tab) => {

```

```

const isActive = currentView === tab.id;

return (
  <button
    key={tab.id}
    onClick={() => (tab.id === 'home' || tab.id === 'library') && setView(tab.id)}
    className={`flex flex-col items-center justify-center gap-0.5 transition-all
duration-300 relative ${
      isActive ? 'text-blue-500' : 'text-gray-500 active:text-gray-300'
    }}
    >
    {/* Indicador de brillo sutil para la pestaña activa */}
    {isActive && (
      <div className="absolute -top-1 w-1 h-1 bg-blue-500 rounded-full shadow-
[0_0_8px_rgba(59,130,246,0.8)]" />
    )}

    <tab.icon
      size={20}
      strokeWidth={isActive ? 2.5 : 2}
      className="transition-transform active:scale-90"
    />

    <span className="text-[9px] font-black uppercase tracking-tighter">{tab.label}</
span>
  </button>
);
}}
</nav>
);
};

```

```

import { Plus, Clock3, Play, Pause, Heart, Trash2 } from 'lucide-react';
import { usePlayerStore } from '../store/usePlayerStore';

```

```

export const Library = () => {
  const { songs, addSongs, clearSongs, setTrack, currentTrack, isPlaying, togglePlay } =
    usePlayerStore();

```

```

const handleFileUpload = (e: React.ChangeEvent<HTMLInputElement>) => {

```

```

const files = e.target.files;
if (!files || files.length === 0) return;
addSongs(Array.from(files));
e.target.value = "";
};

```

```

return (
  <div className="flex flex-col h-full animate-view-entry">
    {/* 1. CABECERA CON GLASSMORPHISM SUTIL */}
    <header className="p-6 md:p-10 flex flex-col md:flex-row justify-between items-start
md:items-end gap-6 bg-gradient-to-b from-[var(--glass-bg)] to-transparent">
      <div>
        <h2 className="text-4xl md:text-7xl font-black tracking-tighter text-[var(--text-
main)]">
          Library
        </h2>
        <p className="text-[var(--text-muted)] font-medium mt-2 text-sm md:text-base
uppercase tracking-widest">
          {songs.length} Tracks en el dispositivo
        </p>
      </div>

      <div className="flex w-full md:w-auto gap-3">
        <label className="flex-1 md:flex-none flex items-center justify-center gap-3 px-8 py-4
bg-[var(--text-main)] text-[var(--bg-main)] rounded-full font-black cursor-pointer
hover:scale-105 transition-all shadow-xl active:scale-95">
          <Plus size={20} strokeWidth={3} />
          <span>ADD MUSIC</span>
          <input
            type="file"
            multiple
            accept="audio/*"
            className="hidden"
            onChange={handleFileUpload}
          />
        </label>

        {songs.length > 0 && (
          <button
            onClick={clearSongs}

```

```

        className="p-4 text-red-500 bg-red-500/10 rounded-full hover:bg-red-500/20
border border-red-500/20 transition-all"
      >
        <Trash2 size={20} />
      </button>
    )}
  </div>
</header>

```

```

{ /* 2. LISTA DE CANCIONES */
<div className="px-4 md:px-10 flex-1">
  { /* Cabecera de la tabla */
    <div className="grid grid-cols-[40px_1fr_40px] md:grid-cols-[50px_1fr_120px_50px]
gap-4 px-4 py-3 text-[var(--text-muted)] text-[10px] uppercase tracking-[0.3em] font-black
border-b border-[var(--glass-border)]">
      <span className="text-center">#</span>
      <span>Título / Artista</span>
      <span className="hidden md:flex justify-end pr-4 text-sm font-mono">
        <Clock3 size={14} />
      </span>
      <span></span>
    </div>

```

```

    { /* LISTADO: El padding inferior 'pb-72' asegura que el scroll no choque con la PlayerBar
+ MobileTabs */
    <div className="flex flex-col mt-4 gap-1 pb-72 md:pb-40">
      {songs.length === 0 ? (
        <div className="py-24 text-center border-2 border-dashed border-[var(--glass-
border)] rounded-[40px] flex flex-col items-center gap-4">
          <p className="text-[var(--text-muted)] font-medium">
            Sube archivos MP3 para empezar.
          </p>
        </div>
      ) : (
        songs.map((track, index) => {
          const isSelected = currentTrack?.id === track.id;

          return (
            <div
              key={track.id}

```

```

onClick={() => (isSelected ? togglePlay() : setTrack(track))}
className={`group grid grid-cols-[40px_1fr_40px] md:grid-cols-
[50px_1fr_120px_50px] items-center gap-4 px-4 py-3 rounded-2xl transition-all border
border-transparent cursor-pointer ${
  isSelected
    ? 'bg-blue-600/10 border-[var(--glass-border)]'
    : 'hover:bg-[var(--glass-bg)]'
}`}
>
  { /* Número / Indicador */ }
  <div className="flex justify-center items-center relative w-6 h-6 mx-auto">
    {isSelected && isPlaying ? (
      <div className="flex gap-0.5 items-end h-3 group-hover:opacity-0 transition-
opacity">
        <div className="w-1 bg-blue-500 animate-bounce [animation-duration:0.6s]" /
      >
        <div className="w-1 bg-blue-500 animate-bounce [animation-duration:0.9s]" /
      >
        <div className="w-1 bg-blue-500 animate-bounce [animation-duration:0.7s]" /
      >
    </div>
    ) : (
      <span
        className={`text-xs font-mono transition-opacity ${isSelected ? 'text-
blue-500' : 'text-[var(--text-muted)] group-hover:opacity-0'}`}
      >
        {index + 1}
      </span>
    )}
    <div
      className={`absolute inset-0 flex items-center justify-center opacity-0 group-
hover:opacity-100 ${isSelected ? 'opacity-100' : ''}`}
    >
      {isSelected && isPlaying ? (
        <Pause size={16} fill="currentColor" />
      ) : (
        <Play size={16} fill="currentColor" />
      )}
    </div>
  </div>
</div>

```



```

    {/* Info Canción */}
    <div className="flex items-center gap-4 overflow-hidden">
      <img
        src={track.coverUrl}
        className="w-12 h-12 md:w-11 md:h-11 rounded-xl object-cover bg-white/5
border border-[var(--glass-border)]"
        alt=""
      />
      <div className="truncate">
        <p
          className={`text-sm md:text-base font-bold truncate ${isSelected ? 'text-
blue-500' : 'text-[var(--text-main)]'}`}
        >
          {track.title}
        </p>
        <p className="text-[10px] md:text-[11px] text-[var(--text-muted)] font-semibold
uppercase tracking-widest truncate group-hover:text-[var(--text-main)] transition-colors">
          {track.artist}
        </p>
      </div>
    </div>

    {/* Duración (Mono y grande como pediste) */}
    <span className="hidden md:block text-sm font-mono text-right pr-4 tracking-
tighter text-[var(--text-muted)]">
      --:--
    </span>

    {/* Favorito */}
    <button
      className={`flex justify-end transition-all ${isSelected ? 'text-blue-500' : 'text-
[var(--text-muted)] opacity-0 group-hover:opacity-100 hover:text-red-500'}`}
    >
      <Heart size={18} fill={isSelected ? 'currentColor' : 'none'} />
    </button>
  </div>
);
})
})

```

</div>

</div>

</div>

);

};